

**INSTITUTO
FEDERAL**
Santa Catarina

Programação Funcional em JavaScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

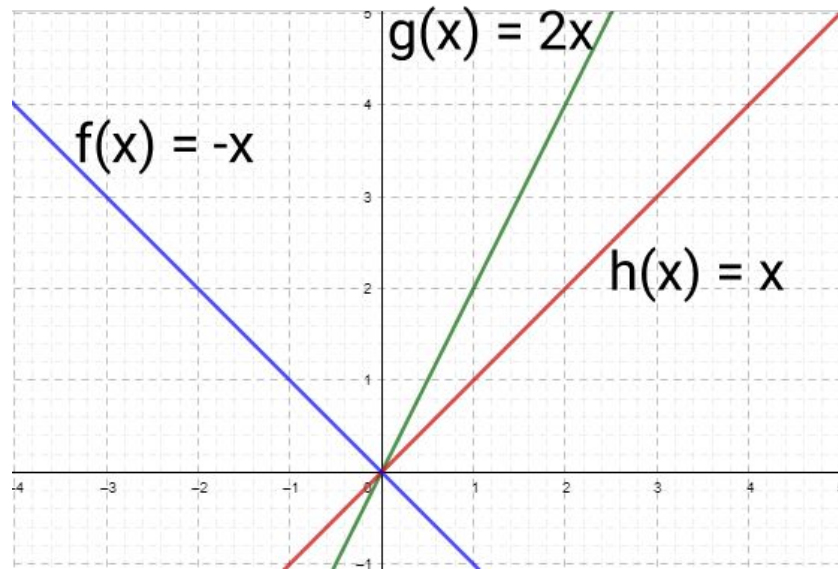
Roboto Mono



Programação Funcional

o que é

- computação como a avaliação de funções matemáticas
- prioriza a declaração de **o que fazer** em vez de como fazer
- foco em
 - funções puras
 - imutabilidade
 - composição de funções



Programação Funcional

vantagens

- previsibilidade e facilidade de depuração
- imutabilidade
- modularidade e reutilização
- concorrência

desafios

- curva de aprendizado
- desempenho
- dificuldade com efeitos colaterais



Programação Funcional

funções puras

■ determinismo

- mesmas entradas, mesmos resultados

■ sem efeitos colaterais

- não altera nenhum estado fora do seu escopo

exemplos

```
let multiplicador = 2;
```

```
// função impura  
function dobrarImpuro(numero) {  
  return numero * multiplicador;  
}
```

```
// função pura  
function dobrarPuro(numero) {  
  return numero * 2;  
}
```



Programação Funcional

funções puras

■ determinismo

- mesmas entradas, mesmos resultados

■ sem efeitos colaterais

- não altera nenhum estado fora do seu escopo

exemplos

```
const numeros = [4, 3, 2, 1];
```

```
const ordenados = numeros.sort( (a, b) => a - b );  
// a função sort() é sempre impura!  
// ela modifica o array original
```

```
const dobro = numeros.map( n => n * 2 );  
// aqui, map() é pura  
// não modifica o array original,  
// nem depende ou altera um estado externo
```

```
let multiplicador = 2;  
const dobro = numeros.map( n => n * multiplicador );  
// aqui, map() é impura  
// depende de um estado externo
```



Programação Funcional

funções puras

■ determinismo

- mesmas entradas, mesmos resultados

■ sem efeitos colaterais

- não altera nenhum estado fora do seu escopo

exemplos

```
// somar os elementos
let numeros = [4, 3, 2, 1];

// programação imperativa
let soma = 0;

for (let i = 0; i < numeros.length; i++) {
  soma += numeros[i];
}

console.log(soma);

// programação funcional
let soma = numeros.reduce( (acc, n) => acc + n, 0 );

console.log(soma);
```



Programação Funcional

funções de ordem superior

■ funções que operam sobre funções

- funções como argumentos

exemplos

```
const executar = (a, b, operacao) => operacao(a, b);
```

```
const somar = (x, y) => x + y;  
const multiplicar = (x, y) => x * y;
```

```
executar(5, 10, somar);  
executar(5, 10, multiplicar);
```



Programação Funcional

funções de ordem superior

- **funções que operam sobre funções**
 - funções como argumentos

exemplos

```
// encontrar os pares
const numeros = [4, 3, 2, 1];

// programação imperativa
const pares = [];

for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    pares.push(numeros[i]);
  }
}

// programação funcional
const ehPar = numero => numero % 2 === 0;

const pares = numeros.filter(ehPar);

// alternativa mais direta
const pares = numeros.filter( x => x % 2 === 0 );
```



Programação Funcional

funções de ordem superior

■ funções que operam sobre funções

- funções como argumentos
- retornam funções

exemplos

```
const somar = a => b => a + b;  
// const somar = a => { return b => a + b };
```

```
const somarCinco = somar(5);  
// const somarCinco = b => 5 + b;
```

```
console.log(somarCinco(10)); // 15
```

```
const somarDez = somar(10);  
// const somarDez = b => 10 + b;
```

```
console.log(somarDez(7)); // 17
```



Programação Funcional

funções de ordem superior

- **funções que operam sobre funções**
 - funções como argumentos
 - retornam funções
- **funções são cidadãos de 1ª classe**

exemplos

```
let senha = 'minhasenha';
let nomeUsuario = 'jose';

// validação imperativa
const validar = (valor, min, max) => {
  return valor.length >= min && valor.length <= max;
}
validar(senha, 8, 16);
validar(nomeUsuario, 4, 12);

// validação funcional
const validar = (min, max) => {
  return (valor) => valor.length >= min &&
    valor.length <= max;
};

const validarSenha = validar(8, 16);
const validarNomeUsuario = validar(4, 12);

validarSenha(senha);
validarNomeUsuario(nomeUsuario);
```



Programação Funcional

currying

- transformação de uma função com vários parâmetros em uma sequência de funções com um parâmetro cada
- os argumentos podem ser "fornecidos aos poucos", criando funções cada vez mais específicas

exemplos

```
// Função convencional
function multiplicar(a, b) {
  return a * b;
}
```

```
// Função com currying
function multiplicar(a) {
  return function(b) {
    return a * b;
  };
}
```

```
multiplicar(3)(10); // 30
```

```
const triplica = multiplicar(3);
triplica(10);
```



Programação Funcional

currying

■ vantagens

- cria funções especializadas a partir de uma função genérica
- economiza esforço em escrever novas funções

■ desvantagens

- pode tornar o código visualmente mais complexo e confuso
- perda de clareza

exemplos

```
// Função convencional
function estilizar(cor, tamanho, fonte, elemento) {
  elemento.style.color = cor;
  elemento.style.fontSize = tamanho;
  elemento.style.fontFamily = fonte;
}
```



Programação Funcional

currying

■ vantagens

- cria funções especializadas a partir de uma função genérica
- economiza esforço em escrever novas funções

■ desvantagens

- pode tornar o código visualmente mais complexo e confuso
- perda de clareza

exemplos

```
// Função com currying
function estilizar(cor) {
  return function (tamanho) {
    return function (fonte) {
      return function (elemento) {
        elemento.style.color = cor
        elemento.style.fontSize = tamanho
        elemento.style.fontFamily = fonte
      }
    }
  }
}

const estilizarAlerta = estilizar('red')('20px')('Arial')

const elemento = document.querySelector('#meuTexto')
estilizarAlerta(elemento)
```



Programação Funcional

recursão

- função que invoca a si mesma repetidamente até atingir uma condição de parada
- pode ser dividida em duas partes
 - caso base
 - passo indutivo
- equivalente à indução matemática
- adequado para estruturas recursivas
 - árvores/DOM
 - menus
 - JSON aninhado

exemplos

```
// função iterativa
let fatorial = numero => {
  let resultado = 1;

  for (let i = 1; i <= numero; i++) {
    resultado *= i;
  }

  return resultado;
}

// função recursiva
function fatorial (n) {

  if (n === 0 || n === 1) return 1;
  return n * fatorial(n - 1);
}
```

Programação Funcional

recursão

■ vantagens

- simplificação de problemas complexos
- clareza e elegância

■ desvantagens

- custo de desempenho
- dificuldade de entendimento
- comparação com loops
- pode causar estouro de pilha em recursões muito profundas (*stack overflow*)

exemplos

```
// função recursiva para imprimir todos os elementos html
function percorrerDOM(elemento) {

    if (elemento.tagName) {
        console.log(elemento.tagName);
    }

    let filhos = Array.from(elemento.children);

    filhos.forEach(filho => { percorrerDOM(filho); });
}
```



Programação Funcional

memoization

- "memória" de cálculos anteriores
- armazena os resultados de chamadas anteriores
- melhora o desempenho de funções
 - otimiza algoritmos recursivos
- **desvantagem** consumo de memória

exemplos

```
// função iterativa
let fatorial = numero => {
  let resultado = 1;

  for (let i = 1; i <= numero; i++) {
    resultado *= i;
  }

  return resultado;
}

// função recursiva
function fatorial (n) {

  if (n === 0 || n === 1) return 1;
  return n * fatorial(n - 1);
}
```



Programação Funcional

memoization

- "memória" de cálculos anteriores
- armazena os resultados de chamadas anteriores
- melhora o desempenho de funções
 - otimiza algoritmos recursivos
- **desvantagem** consumo de memória

exemplos

```
// utilizando memoization para calcular fatorial
function fatorialMemo() {
  let cache = {};

  return function fatorial(n) {
    if (n in cache) return cache[n];

    if (n === 0 || n === 1) return 1;

    cache[n] = n * fatorial(n - 1);
    return cache[n];
  };
}

const fatorial = fatorialMemo();

console.log(fatorial(5)); // 120
console.log(fatorial(5)); // 120 (cache)
```



Programação Funcional

closure

- conjunto composto pelo objeto função e variáveis capturadas do escopo léxico
- a função "lembra" do ambiente onde foi criada
- ocorre quando funções são criadas dentro de outras funções
- a função interna usa variáveis da função externa.

exemplos

```
const x = 10;

function fora() {
  const y = 20;

  function dentro() {
    return x + y;
  }

  return dentro;
}

const funcaoDentro = fora();

console.log(funcaoDentro()); // 30
```



Programação Funcional

closure

■ vantagens

- protegem dados e detalhes da implementação (encapsulamento)
- mantém o estado sem depender de variáveis globais ou objetos externos

■ desvantagens

- dificuldade de depuração
- uso excessivo de memória
- comportamento inesperado em loops

exemplos

```
function fatorialMemo() {  
  let cache = {};  
  
  return function fatorial(n) {  
    if (n in cache) return cache[n];  
  
    if (n === 0 || n === 1) return 1;  
  
    cache[n] = n * fatorial(n - 1);  
    return cache[n];  
  };  
}  
  
const fatorial = fatorialMemo();  
  
console.log(fatorial(5)); // 120  
console.log(fatorial(5)); // 120 (cache)
```



Programação Funcional

closure

■ vantagens

- protegem dados e detalhes da implementação (encapsulamento)
- mantém o estado sem depender de variáveis globais ou objetos externos

■ desvantagens

- dificuldade de depuração
- uso excessivo de memória
- comportamento inesperado em loops

exemplos

```
function criaContador() {  
  let contador = 0;  
  
  return function() {  
    contador++;  
    console.log(contador);  
  };  
}  
  
let contadorClique = criaContador();  
  
const botao = document.getElementById("botao");  
botao.addEventListener("click", contadorClique);
```



Composição de Funções

composição de funções

- funções complexas a partir de funções simples
- o resultado de uma função é passado como entrada para outra
 - $f(g(x))$

exemplos

```
// composição manual
const dobrar = x => x * 2;
const somar = x => x + 1;

let resultado = somar(dobrar(5));

// composição por encadeamento de funções
function somaParesAoQuadrado(array) {
  return array
    .filter(numero => numero % 2 === 0)
    .map(numero => numero ** 2)
    .reduce((acc, numero) => acc + numero, 0);
}
```



Composição de Funções

composição de funções

■ vantagens

- cada função realiza uma única operação (modularidade)
- funções pequenas pode ser reaplicadas em outras situações
- mais fáceis de manter e modificar

■ desvantagens

- dificuldade de depuração
- baixa legibilidade
- dificuldade com efeitos colaterais

exemplos

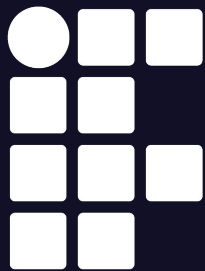
```
// pipe clássico
const ePar = n => n % 2 === 0;
const quadrado = n => n ** 2;
const soma = (a, b) => a + b;

const pipe = (...funcoes) => {
  return x => funcoes.reduce((v, f) => f(v), x);
};

const somaParesAoQuadrado = pipe(
  arr => arr.filter(ePar),
  arr => arr.map(quadrado),
  arr => arr.reduce(soma, 0)
);

somaParesAoQuadrado([1,2,3,4,5,6]); // 56
```





**INSTITUTO
FEDERAL**
Santa Catarina

Programação Funcional em JavaScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas